High Performance Computing: Labs 1 to 3

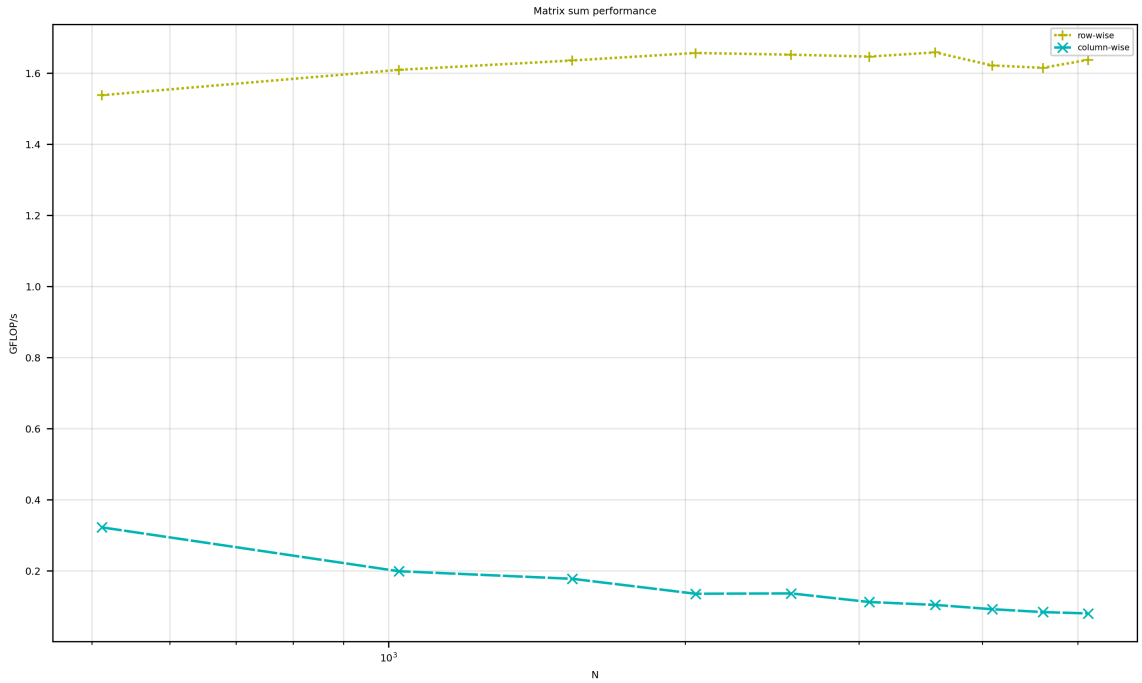
Hacini Malik

March 24, 2026

Matrix norm

The first benchmark isolates a pure memory-layout effect. Both kernels sum the same `N^2` values, but only the row-wise traversal follows contiguous addresses in row-major storage.

When `N` grows, the column-wise version turns a streaming access into a strided one, so cache lines are used poorly and throughput collapses.



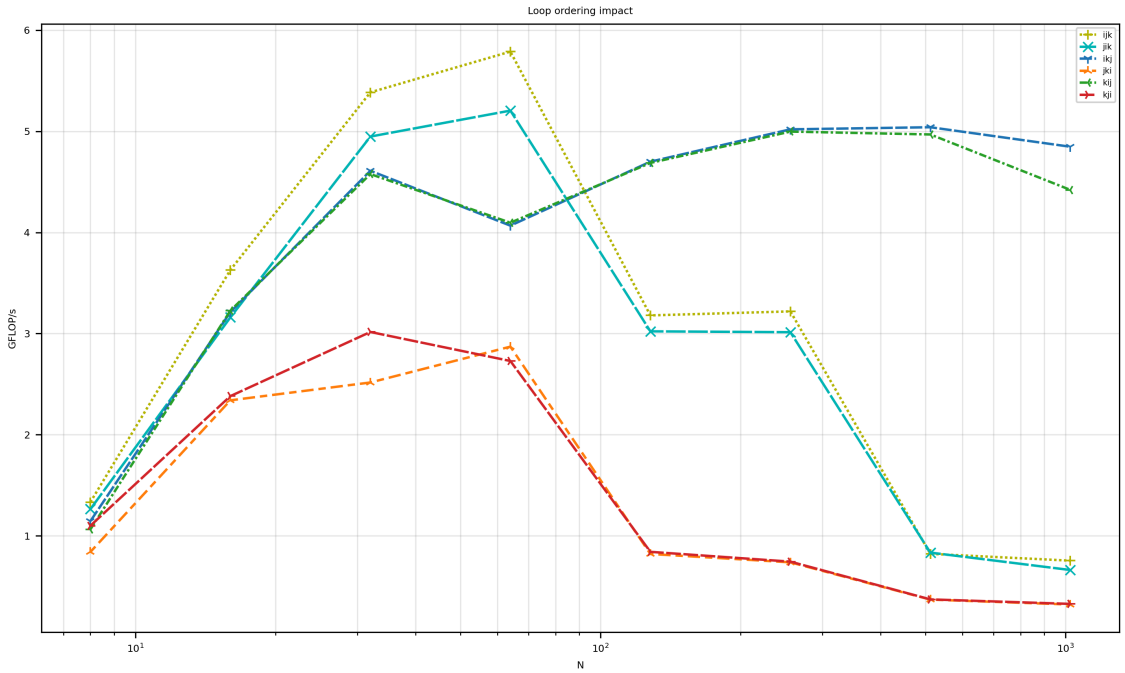
Matrix-matrix multiplication: loop order impact

All six loop orderings perform the same arithmetic, but they reuse memory differently. The matrices are initialized so that

$$AB = I$$

and every kernel can therefore be validated before comparing speed.

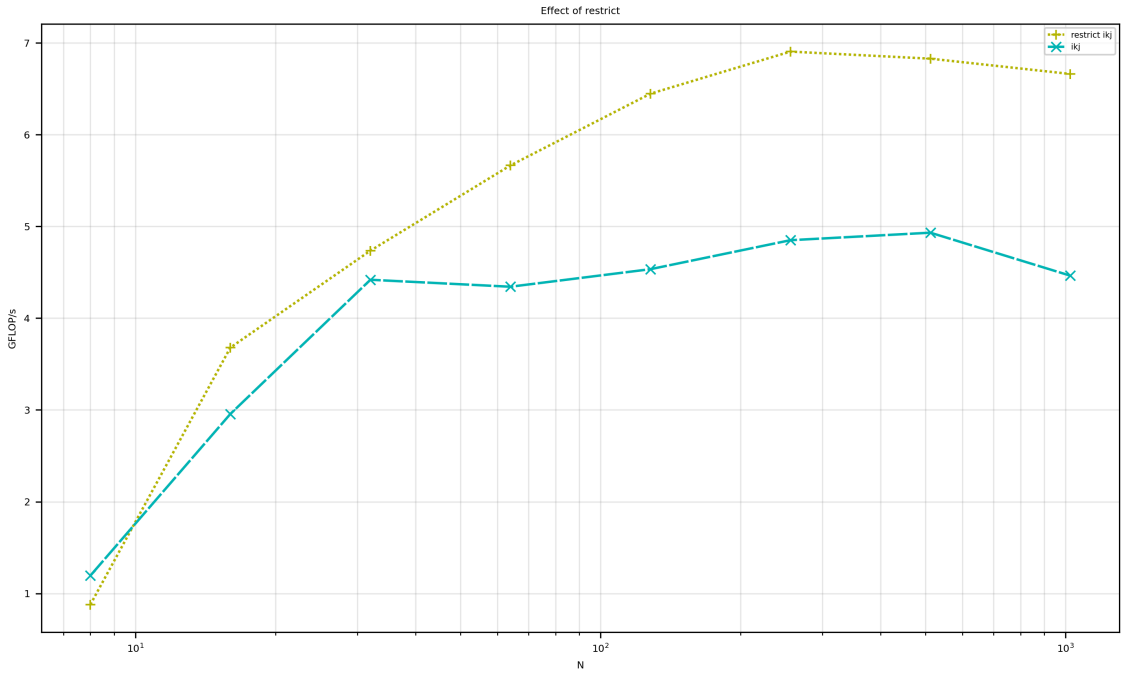
The fastest orderings are the ones whose inner loop streams through `B[k,*]` and `C[i,*]`, because they stream through contiguous memory.



Aliasing and restrict statements

Once `ikj` is identified as the best scalar baseline, the next step is to remove unnecessary aliasing assumptions. Adding `__restrict__` tells the compiler that `A`, `B`, and `C` do not overlap.

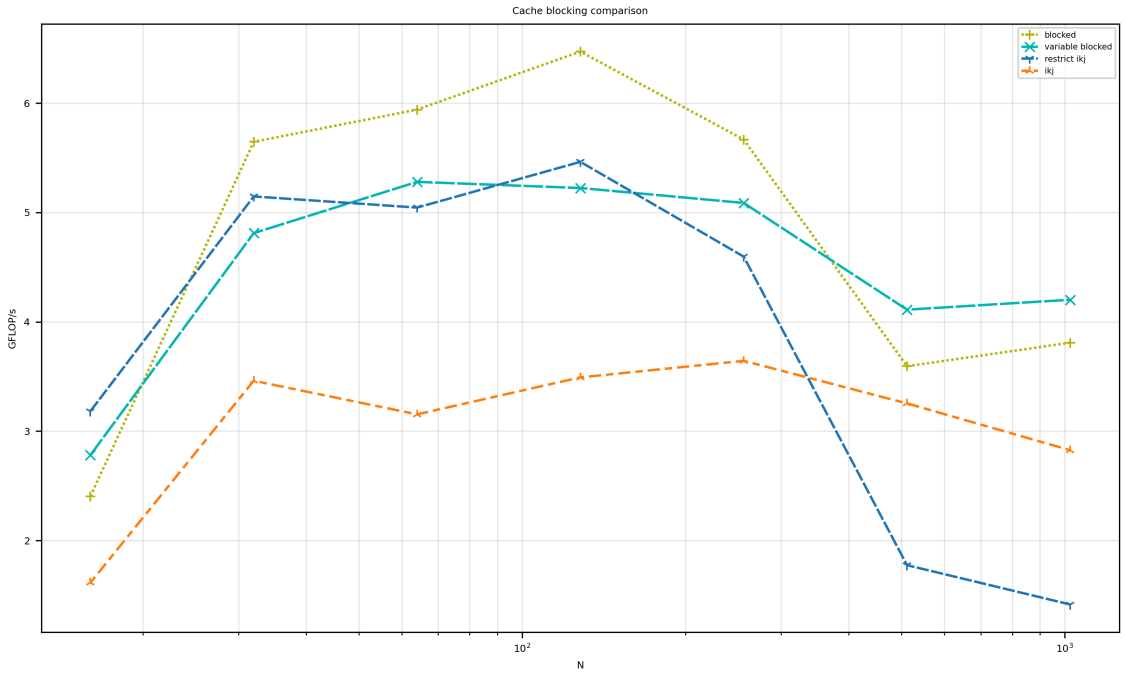
The algorithm does not change, but the compiler can keep values in registers and schedule loads and stores more aggressively, which explains the clear gain on medium and large matrices.



Cache blocking

Lab 2 starts from the restricted `ikj` kernel and reorganizes the computation into tiles. Instead of streaming each matrix once, the blocked version keeps small regions of `A`, `B`, and `C` hot in cache and reuses them before moving on.

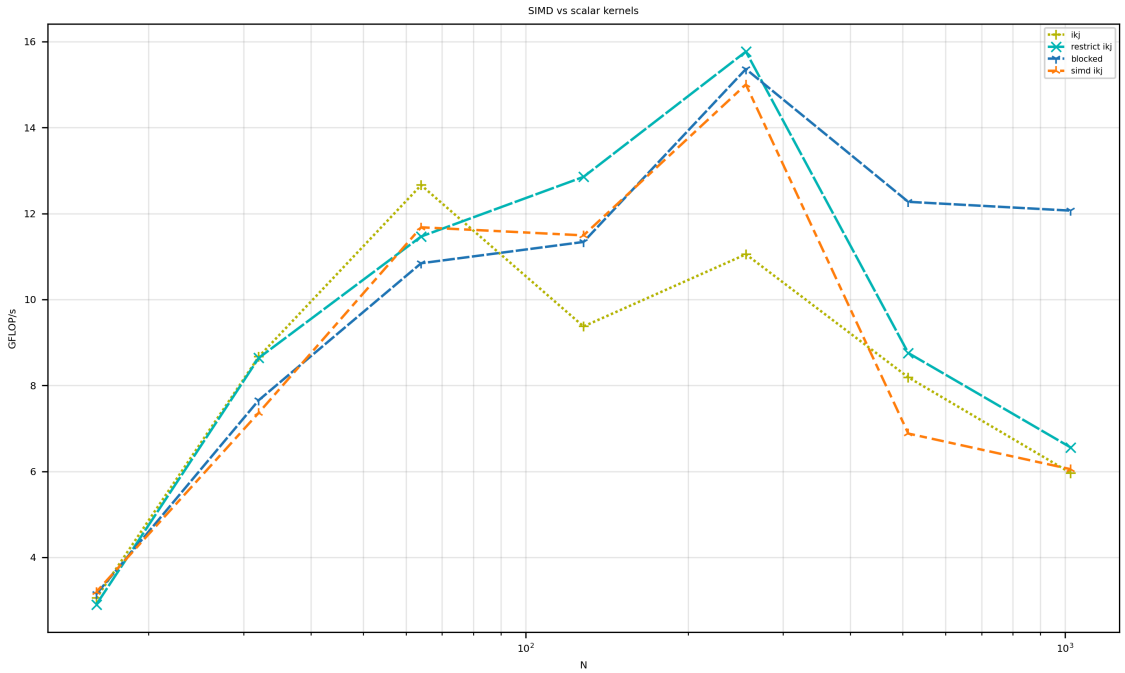
Both variable and constant block sizes were implemented. On this machine, the constant blocked kernel becomes the most reliable choice once the matrices are too large for the simpler scalar variants.



SIMD vectorization

The matrices are then allocated with `posix_memalign(..., 512, ...)`, and the innermost loop is exposed to vectorization with `#pragma omp simd`.

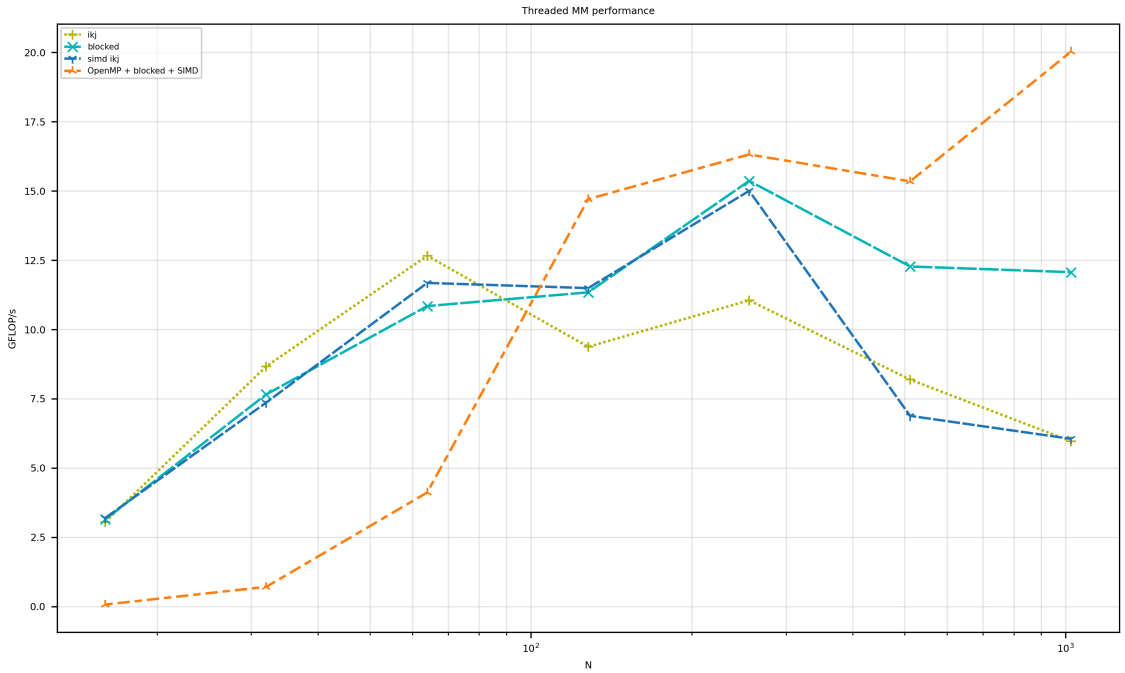
SIMD increases the amount of arithmetic done per cycle, but it does not replace locality. The best results still come from combining aligned data, vectorization, and a cache-aware traversal.



OpenMP threading

Lab 3 extends the blocked matrix multiplication to multi-core execution. The outer row blocks are distributed across threads so that each thread updates a disjoint tile of `C`.

This keeps the kernel race-free while preserving the blocked structure from Lab 2. The gain is modest on small cases but gets significant for larger tasks.



Numerical integration: test problem

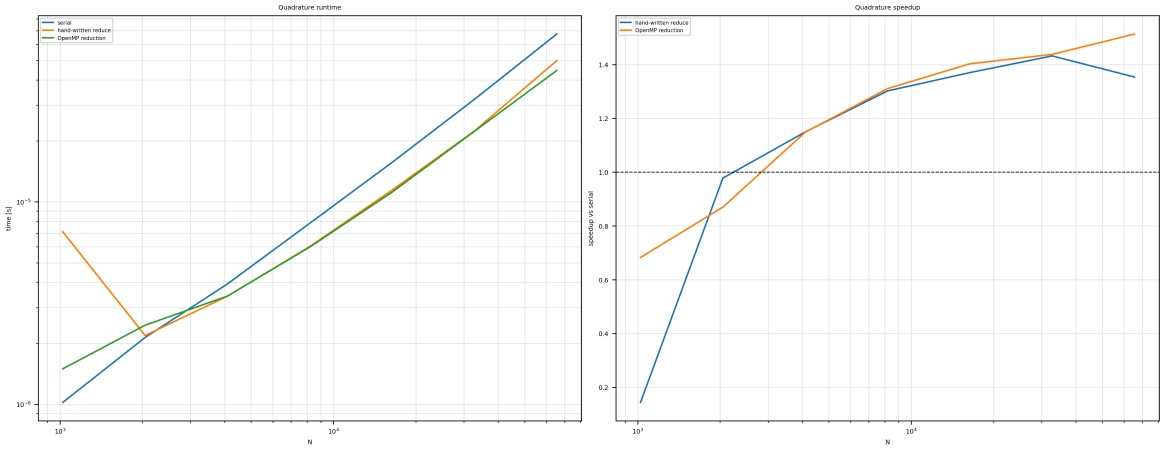
We now move from matrix multiplication to numerical integration. The integral can be esaily analyitcally evaluated :

$$\int_0^1 \frac{4}{1+x^2}\,\mathrm{d}x = \pi$$

On the current machine, the serial midpoint rule reaches the required accuracy of `10^-8` with `N = 4096` intervals. The reduction benchmarks are therefore run around that validated resolution rather than on arbitrary problem sizes.

Numerical integration: reduction benchmark

Two parallel reductions are compared. The first is written by hand with one accumulator per thread and an explicit final sum. The second relies on OpenMP's built-in reduction clause.



Both reproduce the serial midpoint result up to floating-point roundoff, but the OpenMP version is shorter to express and slightly faster on the tested range.

Conclusion

From Lab 1 to Lab 3, the same logic drives every improvement. First choose a traversal that respects the memory layout, then improve reuse with blocking, then increase single-core throughput with SIMD, and finally exploit the available cores with OpenMP.

The numerical-integration benchmark confirms the same broader lesson: parallelism only becomes profitable once the workload is large enough, and the cleanest implementation is often also the most effective one.